

◆ CS EDUCATION ◆

🔍

CORS

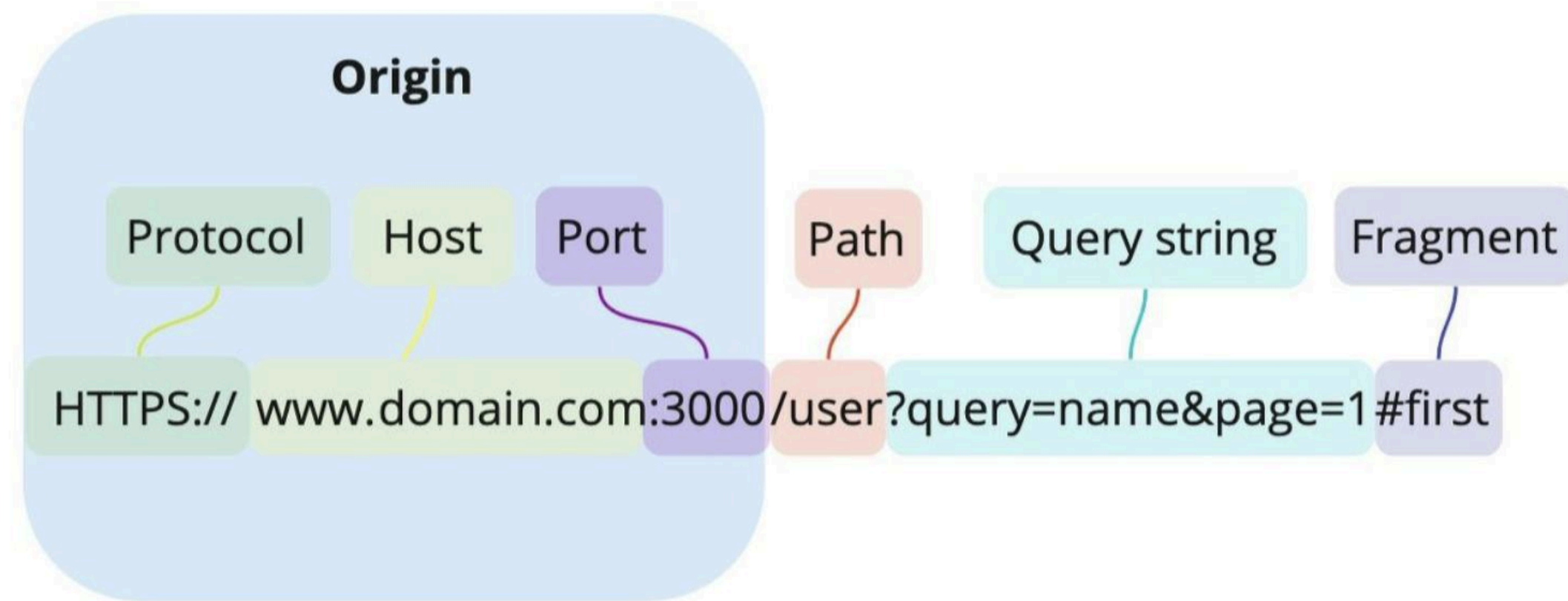
26.07.31
임유미

교육 목차

1. 웹의 Origin 개념
2. CORS의 개념
3. CORS 시나리오
4. CORS 에러 대응법

웹의 Origin 개념

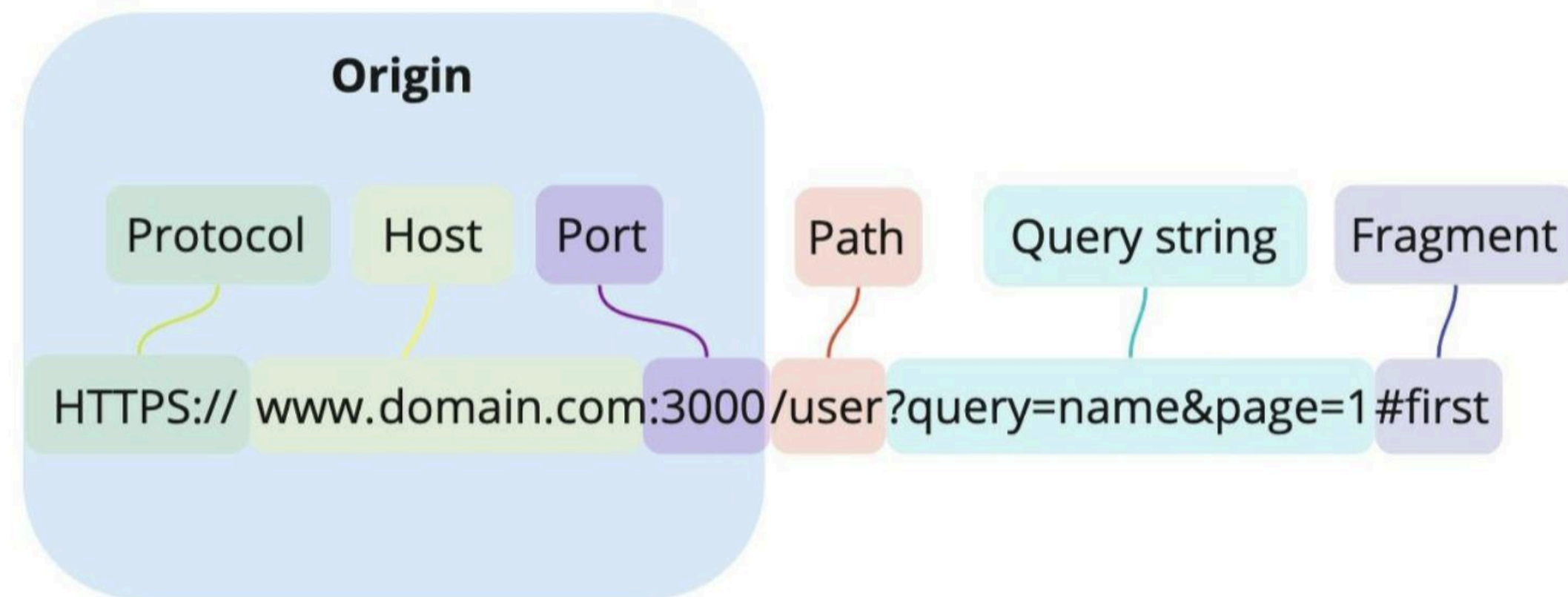
URI의 구성요소



Origin(출처) = Protocol + Host + Port

Origin 중 하나라도 다르다면 다른 출처가 됨

웹의 Origin 개념



Origin 중 하나라도 다르면 **다른 출처**가 됨

예시) `https://cotato.kr`과 `http://cotato.kr` 은 프로토콜이 다르므로 다른 출처

웹의 Origin 개념

기준 출처(Origin): <https://www.cotato.kr/>

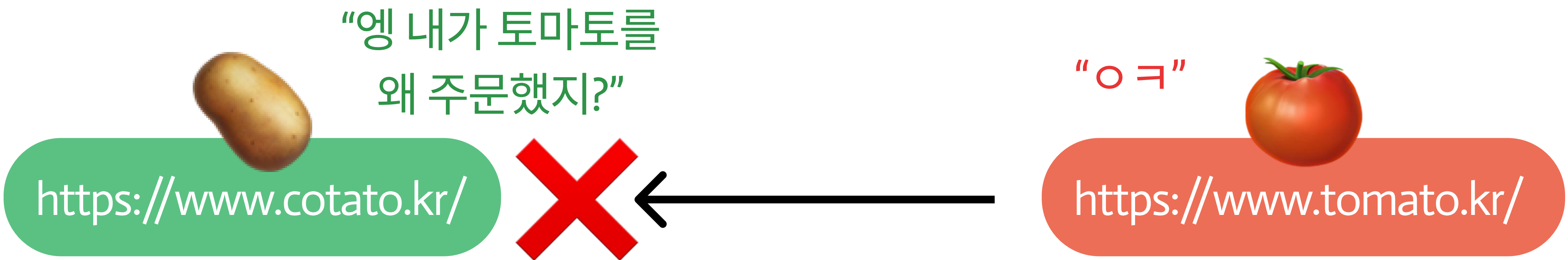
URL	동일한 출처인가?
https://www.cotato.kr:443/	✅ 프로토콜, 도메인, 포트가 같음 HTTPS의 기본 포트인 443은 생략 가능
https://www.cotato.kr/example2/	✅ 경로는 오리진에 포함되지 않아 같음
http://www.cotato.kr/example/	❌ 프로토콜이 다름
https://en.cotato.kr/example/	❌ 도메인이 다름
https://www.cotato.kr:8080/example/	❌ 포트가 다름
http://en.cotato.kr:8080/example/	❌ 프로토콜, 도메인, 포트가 다름

다른 출처에 리소스를 요청하면?



1. 브라우저에서 <https://www.cotato.kr/> 출처를 가진 웹 애플리케이션에서 <https://www.tomato.kr/> 출처의 리소스를 요청함

다른 출처에 리소스를 요청하면?



2. <https://tomato.kr> 에서 응답,
그러나 <https://www.cotato.kr> 출처의 브라우저가
그 응답을 스크립트가 읽지 못하도록 차단함

다른 출처에 리소스를 요청하면?

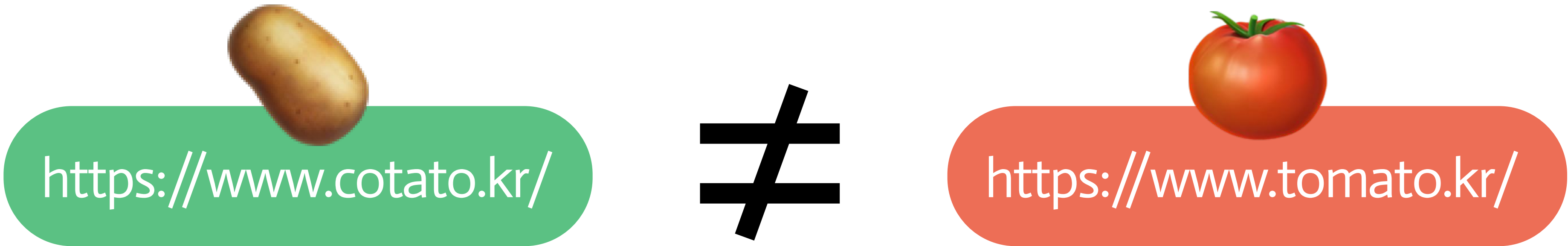


✖ Access to XMLHttpRequest at 'http://localhost:5000/global_config' step1:1 from origin 'http://localhost:8080' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: No 'Access-Control-Allow-Origin' header is present on the requested resource.

3. CORS 에러 발생!!

SOP(Same-Origin Policy)

SOP(Same-Origin Policy, 동일 출처 정책)
: 서로 다른 출처일 때 브라우저가 리소스 요청과 응답을 차단하는 정책



SOP(Same-Origin Policy)

- 브라우저는 기본적으로 다른 서버를 신뢰 X
- 다른 서버에 요청을 보내거나 응답을 받는 걸 차단함

토큰, 쿠키와 같이 민감한 사용자 정보는 브라우저에 저장됨
→ 이 정보를 탈취하면 심각한 보안 이슈가 발생하기 때문

이처럼 서로 다른 서버 리소스는 공유하지 않는 브라우저 정책을
SOP(Same-Origin Policy, 동일 출처 정책)라고 함

웹의 발달과 CORS

과거

- 프론트엔드와 백엔드를 분리하지 않고 한 서버에서 함께 구성
- 모든 처리가 같은 도메인 안에서 이루어짐
- 다른 출처로의 요청은 그 자체로 의심스러운 행위로 간주됨



현재

- 클라이언트에서 API를 직접 호출하는 방식이 보편화됨
- 클라이언트와 API 서버가 서로 다른 도메인에 위치하는 경우가 다수
- 출처가 달라도 안전하게 통신할 새로운 규칙이 필요해짐

웹의 발달과 CORS

출처가 다르더라도

과거

- 프론트엔드와 백엔드를 분리하지 않고 한 서버에서 함께 구성
- 모든 처리가 같은 도메인 안에서 이루어짐
- 다른 출처로의 요청은 그 자체로 의심스러운 행위로 간주됨

현재

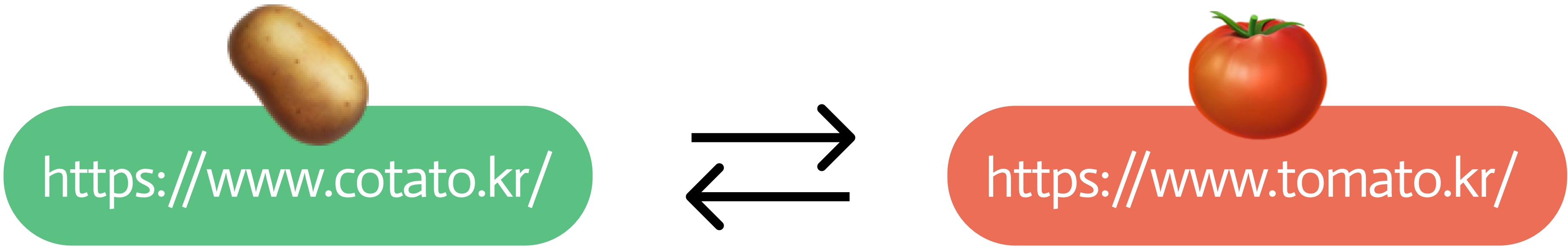
요청·응답을 주고받을 수 있도록

CORS 정책이 등장

- 클라이언트에서 API를 직접 호출하는 방식이 보편화됨
- 클라이언트와 API 서버가 서로 다른 도메인에 위치하는 경우가 다수
- 출처가 달라도 안전하게 통신할 새로운 규칙이 필요해짐

CORS(Cross-Origin Resource Sharing)

CORS(Cross-Origin Resource Sharing, 교차 출처 리소스 공유)
: 서로 다른 출처라도 리소스 요청, 응답을 허용할 수 있도록 하는 정책



CORS(Cross-Origin Resource Sharing)

브라우저가 CORS를 확인하는 방법



http://example.com

Origin: http://example.com



http://api.example.com

1. 웹에서 서버로 리소스를 요청.
이때, 브라우저가 요청 헤더에 **Origin**을 담아 전송

CORS(Cross-Origin Resource Sharing)

브라우저가 CORS를 확인하는 방법



<http://example.com>



Access-Control-Allow-Origin: (비어있음)

2. 서버가 응답 헤더에
Access-Control-Allow-Origin를 포함해서 전송



<http://api.example.com>

***Access-Control-Allow-Origin**: 서버가 리소스 요청을 허용할 출처를 명시하는 응답 헤더

CORS(Cross-Origin Resource Sharing)

브라우저가 CORS를 확인하는 방법



Origin: `http://example.com` ↔ Access-Control-Allow-Origin: (비어있음)

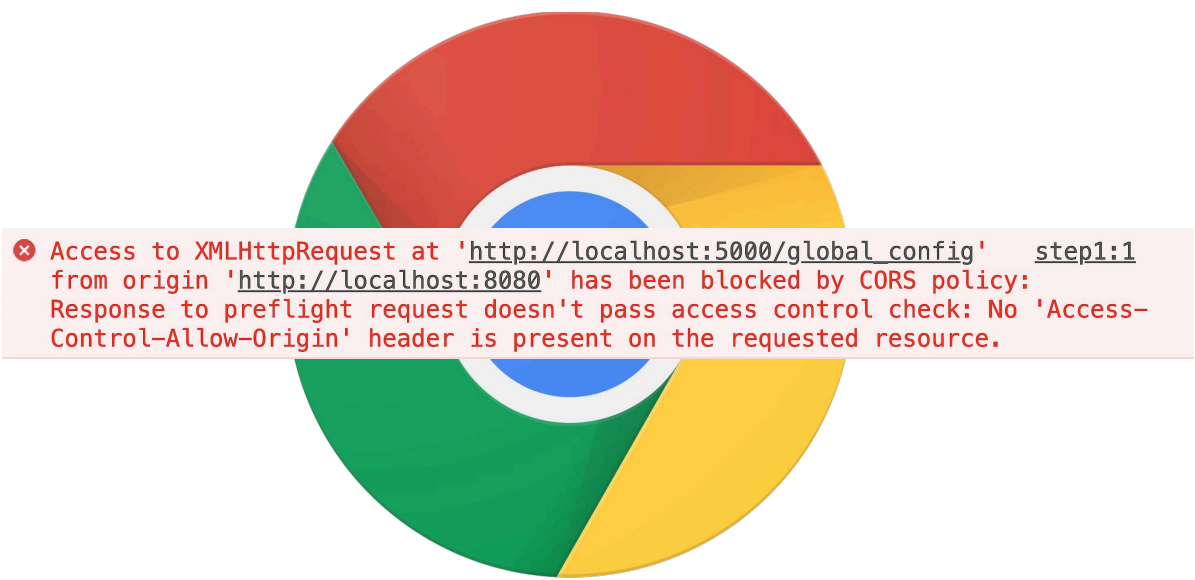
3. 브라우저가 요청시 보낸 Origin과
Access-Control-Allow-Origin 값을 비교

*Access-Control-Allow-Origin: 서버가 리소스 요청을 허용할 출처를 명시하는 응답 헤더

CORS(Cross-Origin Resource Sharing)

브라우저가 CORS를 확인하는 방법

Origin: http://example.com ↔ Access-Control-Allow-Origin: (비어있음)



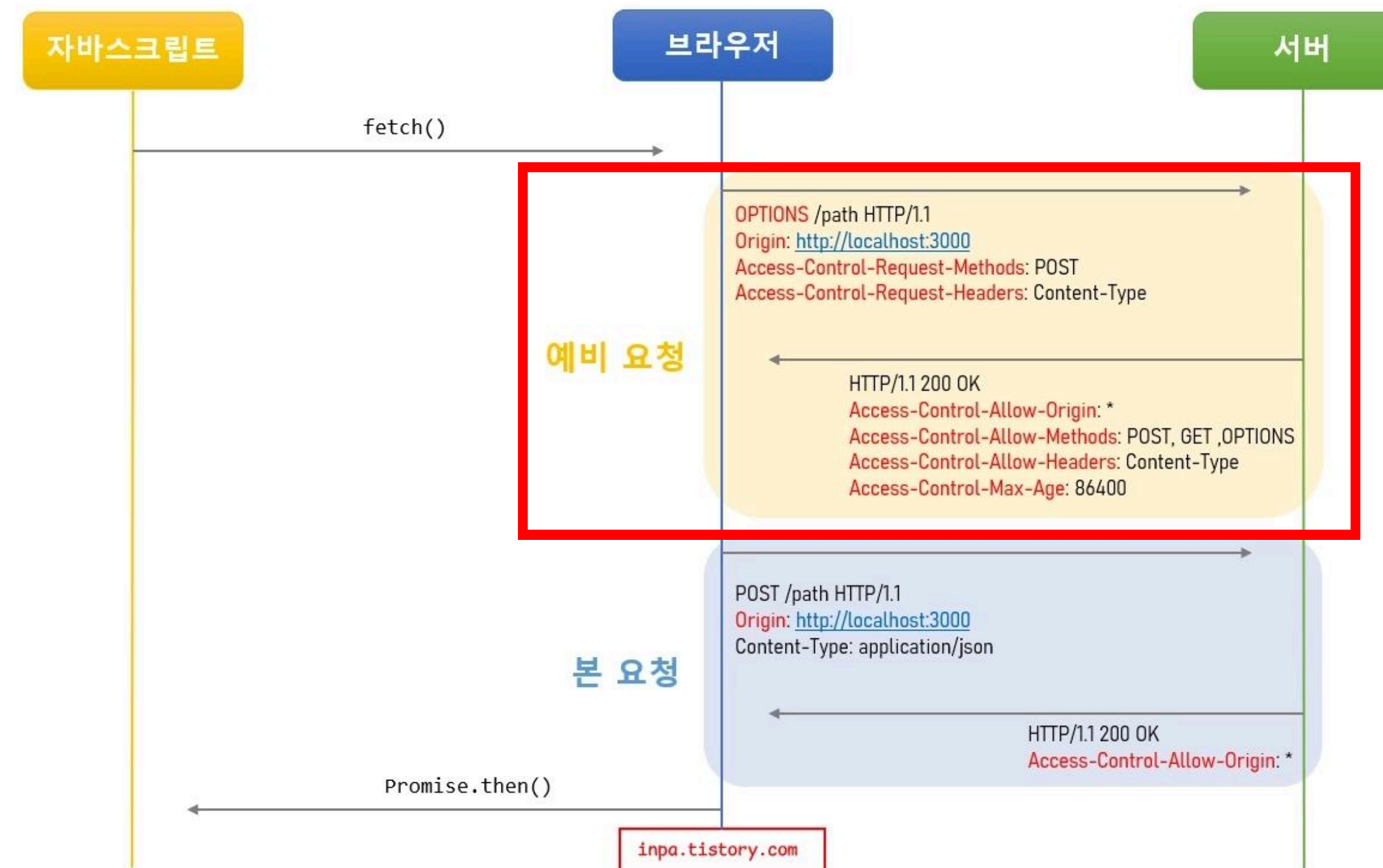
4. 만약 허용 출처 목록 안에 Origin이 포함되지 않는다면
브라우저가 해당 응답을 차단하고
CORS Policy를 위반했다는 에러를 발생시킴

주의 본 요청이 서버까지 도달하는 경우, 클라이언트-서버의 소통은
정상적으로 성공하였으므로 **서버의 로그**에는 **정상 응답**으로 기록됨.
차단은 브라우저에서 일어남.

CORS 시나리오

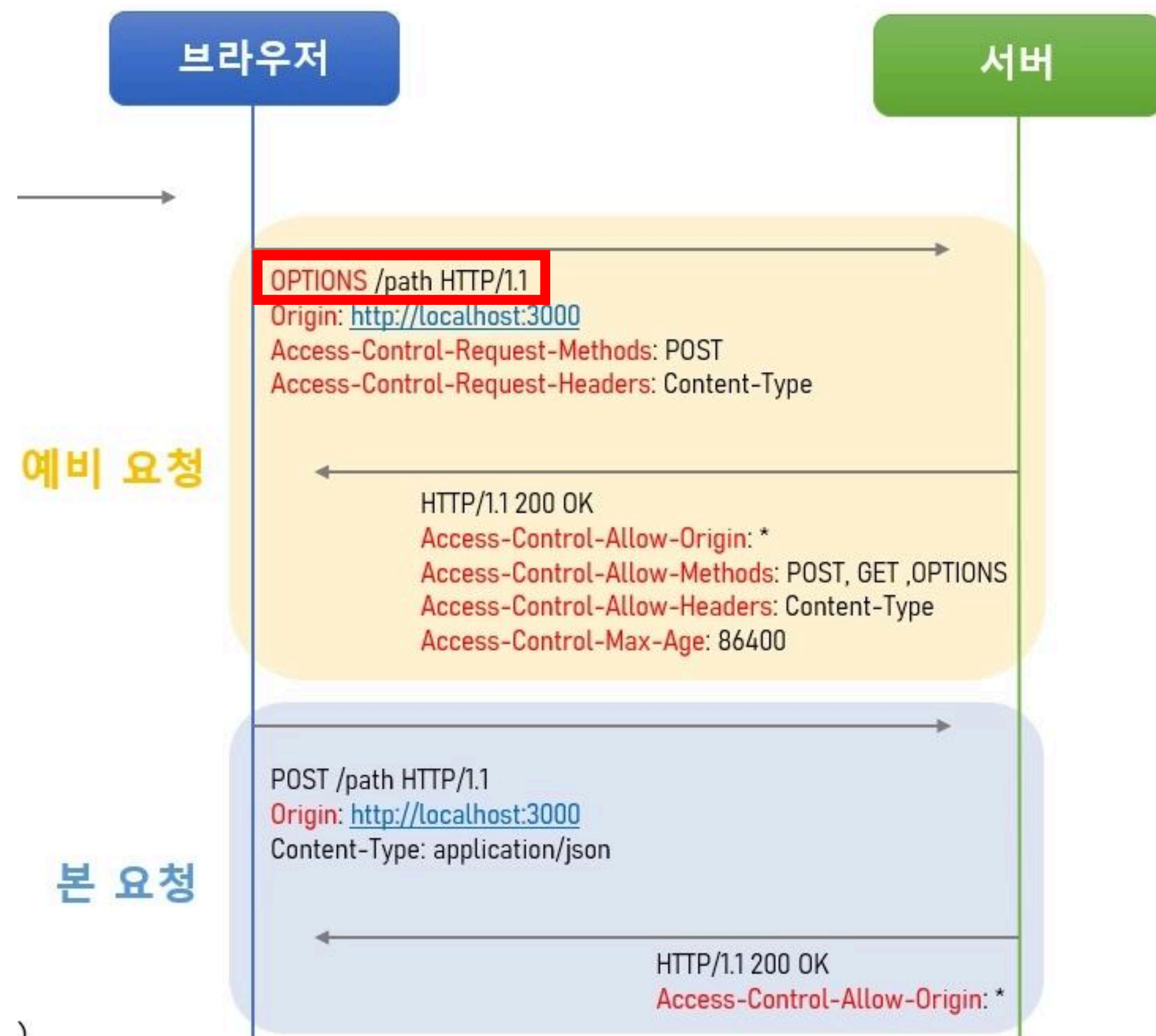
1. 사전 요청(Preflighted requests)
2. 단순 요청(Simple requests)
3. 자격 증명 요청 (Credential Request)

1. 사전 요청(Preflighted requests)



브라우저가 본 요청을 하기 전에 Preflight Request(사전 요청)를 보내 서버와 잘 통신할 수 있는지 확인하는 방식

1. 사전 요청(Preflighted requests)



- 특징: HTTP 메소드가 GET이나 POST가 아닌 **OPTIONS**라는 요청이 사용됨
- 단점: 실제 요청에 걸리는 시간이 늘어나게 되어 어플리케이션 성능에 영향을 미침
- 최적화: **Access-Control-Max-Age**로 캐시될 시간을 명시
- 예시)



최대 2시간

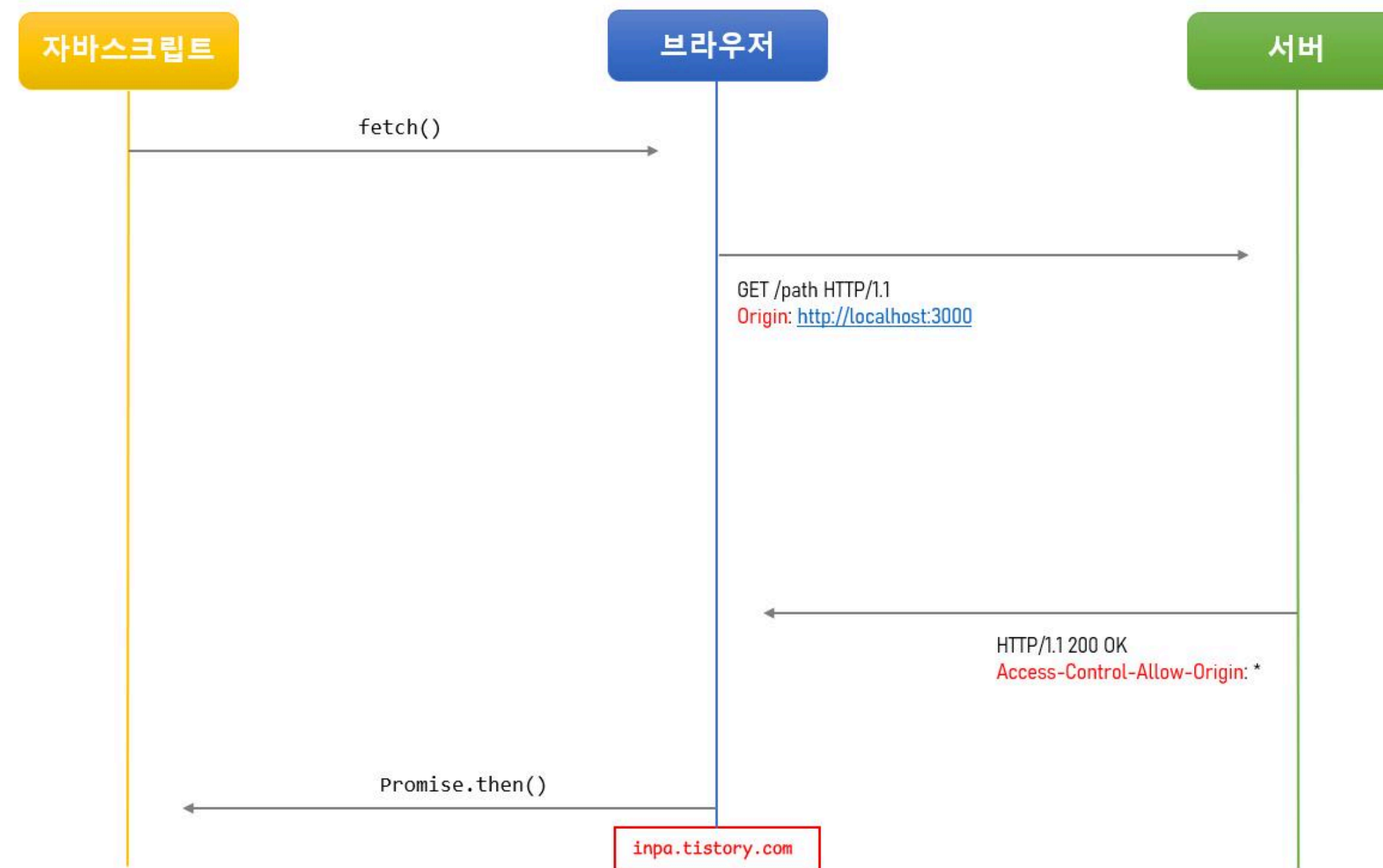


최대 2시간



최대 5분

2. 단순 요청(Simple requests)



사전요청(Preflight)을 생략하고 바로 본 요청을 전송한 후,
서버가 응답 헤더를 보내주면 브라우저가 CORS 정책 위반 여부만 검사하는 방식

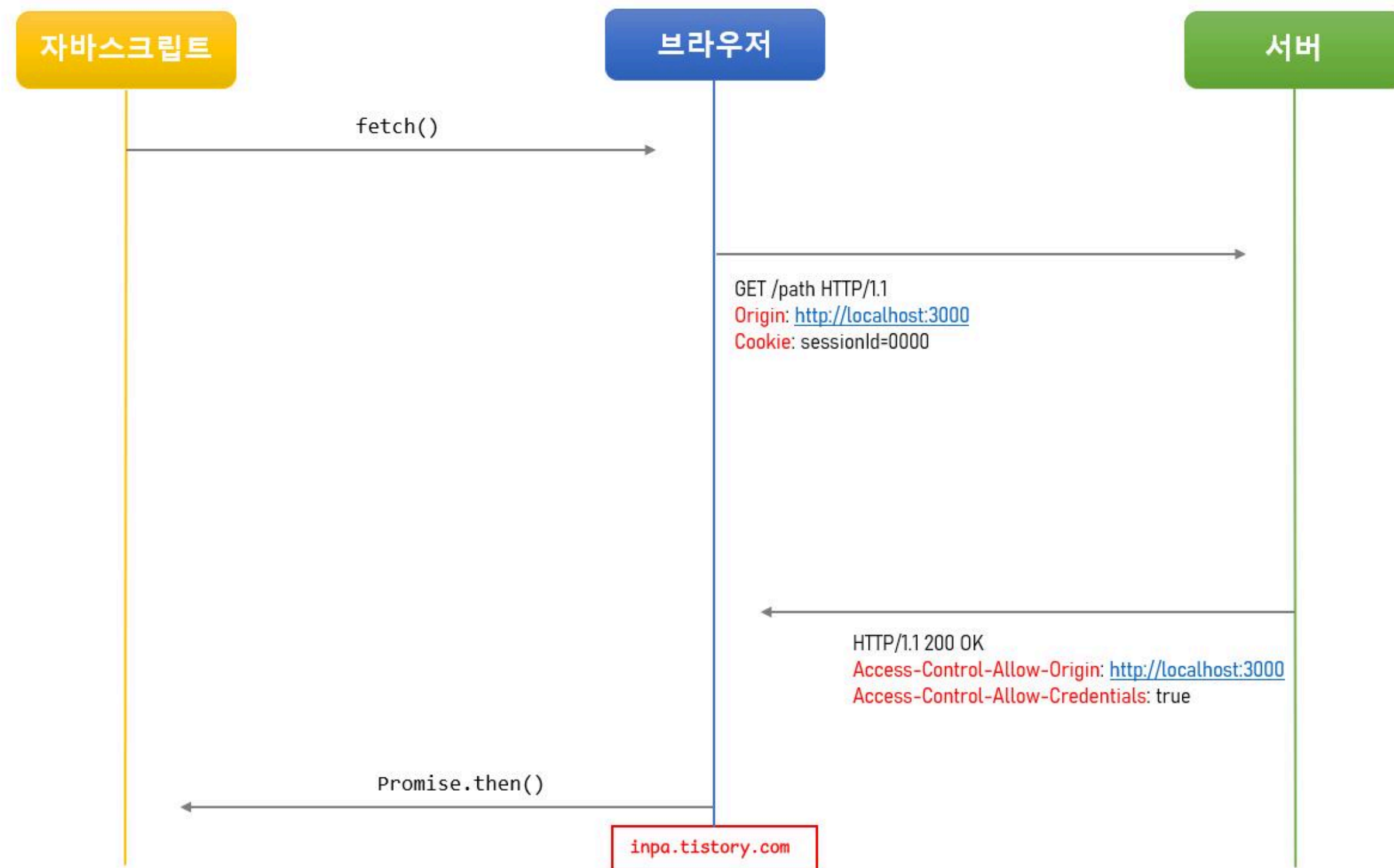
2. 단순 요청(Simple requests)

3가지 경우를 만족할 때만 단순 요청으로 동작

1. 요청의 메소드는 GET, HEAD, POST 중 하나여야 함
2. Accept, Accept-Language, Content-Language, Content-Type, DPR, Dowlink, Save-Data, Viewport-Width, Width 헤더일 경우
3. Content-Type 헤더가 application/x-www-form-urlencoded, multipart/form-data, text/plain 중 하나

위 조건을 모두 만족되어 단순 요청이 일어나는 상황은 드물기 때문에 대부분의 API 요청은 **사전 요청**으로 처리됨

3. 자격 증명 요청 (Credentialed Request)



클라이언트에서 서버에게 자격 인증 정보(Credential)를 실어 요청할때 사용되는 요청

3. 자격 증명 요청 (Credentialed Request)

①클라이언트: credentials 옵션

옵션 값	설명
same-origin(기본값)	같은 출처 요청에만 인증 정보 포함
include	모든 요청에 인증 정보 포함
omit	모든 요청에 인증 정보를 포함하지 않음

```
// fetch 메서드
fetch("https://example.com:1234/users/login", {
  method: "POST",
  credentials: "include",
  // 클라이언트와 서버가 통신할때
  // 쿠키와 같은 인증 정보 값을 공유하겠다는 설정
  body: JSON.stringify({
    userId: 1,
  }),
})
```

- 별도 설정이 없으면 쿠키 등 인증 정보는 절대로 자동 전송되지 않음
- fetch, axios, jQuery 등 다양한 라이브러리로 요청 가능

3. 자격 증명 요청 (Credentialed Request)

② 서버: 응답 헤더 조건

- Access-Control-Allow-Credentials: true 필수
- Access-Control-Allow-Origin,
Access-Control-Allow-Methods,
Allow-Methods, Allow-Headers에 와일드카드(*) 사용 불가

두 조건 중 하나라도 어긋나면 브라우저가 CORS 정책 위반으로 응답을 거부

CORS 에러 대응하기

1. 서버에서 Access-Control-Allow-Origin 응답 헤더 설정

```
Access-Control-Allow-Origin: <origin> | *
```

- 서버에서 요청을 수락할 출처를 명시적으로 지정 가능
 - * 는 모든 출처를 허용하므로 보안에 취약

CORS 에러 대응하기

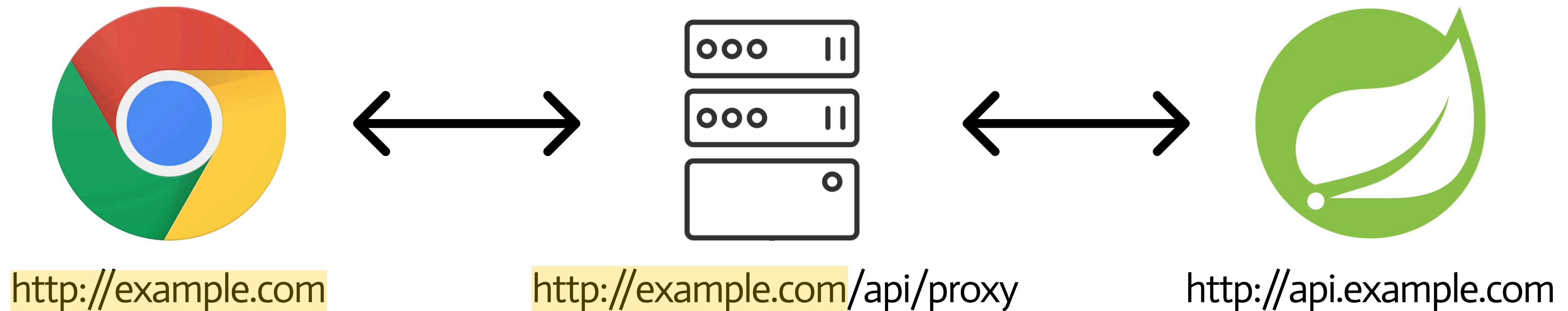
1. 서버에서 Access-Control-Allow-Origin 응답 헤더 설정

```
'Access-Control-Allow-Origin': 'https://myshop.com'
```

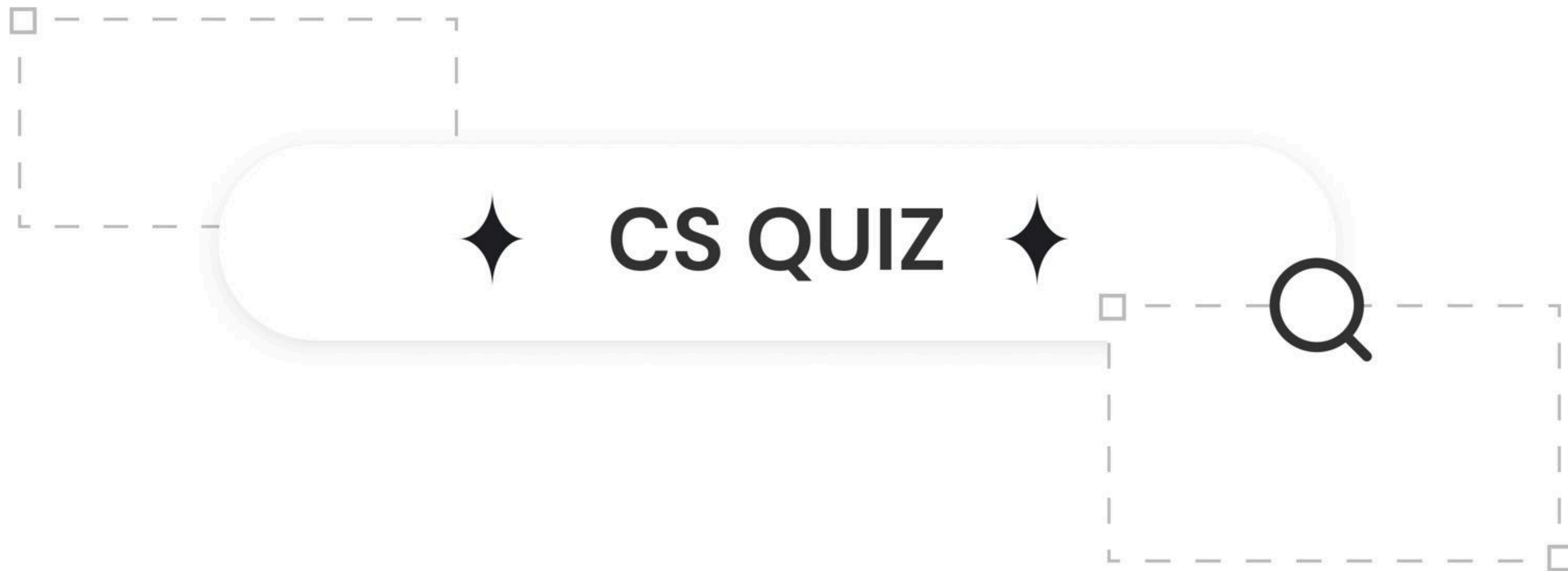
- 리소스 요청을 수락할 출처를 직접 명시하는 방법이 더 안전

CORS 에러 대응하기

2. 프록시 서버 사용



웹 애플리케이션이 리소스를 직접 요청하지 않고,
같은 출처의 프록시 서버가 요청을 대신 전달



Q.1

[객관식]

브라우저가 두 URL을 같은 출처(Origin)로
판단하기 위한 3가지 구성 요소는?

- ① Protocol, Host, Path
- ② Protocol, Host, Port
- ③ Host, Port, Path
- ④ Protocol, Domain, Query String



A.1

[객관식]

브라우저가 두 URL을 같은 출처(Origin)로
판단하기 위한 3가지 구성 요소는?

- ① Protocol, Host, Path
- ② Protocol, Host, Port
- ③ Host, Port, Path
- ④ Protocol, Domain, Query String

Origin은 Protocol + Host + Port로 구성됨.

Path, Query String, Fragment는 Origin 판단에 영향을 주지 않음.

예를 들어 <https://www.cotato.kr/example/>과
<https://www.cotato.kr/example2/>는 경로가 다르지만 동일 출처로 취급됨.

Q.2

[객관식]

SOP(Same-Origin Policy)에 대한 설명으로 옳지 않은 것은?

- A. 동일 출처의 요청만 허용하는 브라우저 정책이다
- B. 서버가 설정하는 정책이다
- C. 토큰·쿠키 등 민감정보 탈취를 막기 위한 정책이다
- D. 다른 출처의 응답을 브라우저가 차단한다



A.2

[객관식]

SOP(Same-Origin Policy)에 대한 설명으로 옳지 않은 것은?

- A. 동일 출처의 요청만 허용하는 브라우저 정책이다
- B. 서버가 설정하는 정책이다
- C. 토큰·쿠키 등 민감정보 탈취를 막기 위한 정책이다
- D. 다른 출처의 응답을 브라우저가 차단한다

SOP는 서버가 아니라 브라우저가 적용하는 정책이다.

Q.3

[순서배열]

브라우저가 CORS를 확인하는 과정을 올바른 순서로 배열하시오.

- (A) 브라우저가 자신이 보낸 Origin과 서버 응답의 허용 출처를 비교한다
- (B) 허용 목록에 없으면 브라우저가 응답을 버리고 CORS 에러를 띄운다
- (C) 브라우저가 요청 헤더에 Origin 정보를 담아 전송한다
- (D) 서버가 응답 헤더에 Access-Control-Allow-Origin 값을 담아 반환한다



A.3

[순서배열]

브라우저가 CORS를 확인하는 과정을 올바른 순서로 배열하시오.

정답 : C → D → A → B

- (C) 브라우저가 요청 헤더에 Origin 정보를 담아 전송한다
- (D) 서버가 응답 헤더에 Access-Control-Allow-Origin 값을 담아 반환한다
- (A) 브라우저가 자신이 보낸 Origin과 서버 응답의 허용 출처를 비교한다
- (B) 허용 목록에 없으면 브라우저가 응답을 버리고 CORS 에러를 띄운다

Q.4

[객관식]

CORS 에러에 대한 설명으로 가장 올바른 것은?

1. 서버가 요청 자체를 거부했기 때문에 발생하는 에러이다
2. 본 요청이 서버까지 도달하는 경우, 서버는 정상적으로 응답했지만 브라우저가 응답을 받은 후 스스로 걸러내는 것이다
3. 서버 로그에는 항상 실패로 기록된다
4. 클라이언트의 네트워크 연결 문제로 발생하는 에러이다



A.4

[객관식]

CORS 에러에 대한 설명으로 가장 올바른 것은?

1. 서버가 요청 자체를 거부했기 때문에 발생하는 에러이다
2. **본 요청이 서버까지 도달하는 경우, 서버는 정상적으로 응답했지만 브라우저가 응답을 받은 후 스스로 걸러내는 것이다**
3. 서버 로그에는 항상 실패로 기록된다
4. 클라이언트의 네트워크 연결 문제로 발생하는 에러이다

서버 입장에서는 요청에 정상 응답했기 때문에 서버 로그에는 성공으로 기록되며, CORS 에러는 브라우저가 응답을 받은 후 스스로 차단하는 것이다.

Q.5

[객관식]

다음은 CORS 시나리오에 대한 설명입니다.
각 설명에 대해 옳지 않은 요청 방식이 매칭된 선지를 고르세요.

- A. 본 요청을 보내기 전에 전송하는 요청으로, 서버와 안전하게 통신할 수 있는지 확인하는 역할을 한다. (사전 요청)
- B. 조건을 모두 만족하는 상황이 드물기 때문에 대부분 다른 방식으로 처리된다. (단순 요청)
- C. 클라이언트가 서버로 리소스를 요청할 때 인증 정보를 함께 전송한다. (자격 증명 요청)
- D. 서버가 보내주는 응답 헤더에 Access-Control-Allow-Credentials: true 를 필수로 설정해야 한다. (단순 요청)



A.5

[객관식]

다음은 CORS 시나리오에 대한 설명입니다.
각 설명에 대해 옳지 않은 요청 방식이 매칭된 선지를 고르세요.

- A. 본 요청을 보내기 전에 전송하는 요청으로, 서버와 안전하게 통신할 수 있는지 확인하는 역할을 한다. (사전 요청)
- B. 조건을 모두 만족하는 상황이 드물기 때문에 대부분 다른 방식으로 처리된다. (단순 요청)
- C. 클라이언트가 서버로 리소스를 요청할 때 인증 정보를 함께 전송한다. (자격 증명 요청)
- D. 서버가 보내주는 응답 헤더에 `Access-Control-Allow-Credentials: true` 를 필수로 설정해야 한다. (단순 요청)

서버가 보내주는 응답 헤더에 `Access-Control-Allow-Credentials: true` 를
필수로 설정해야 하는 요청 방식은 자격 증명 요청이다.

Q.6

[주관식]

사전 요청(Preflight Request) 시
본 요청 전에 브라우저가 사용하는 HTTP 메서드는?



A.6

[주관식]

사전 요청(Preflight Request) 시
본 요청 전에 브라우저가 사용하는 HTTP 메서드는?

정답 : OPTIONS

브라우저는 실제 요청을 보내기 전에 OPTIONS 메서드로 예비 요청을
보내 서버와 통신 가능 여부를 확인한다.

Q.7

[주관식]

fetch의 credentials 옵션 중,
같은 출처 요청에는 인증 정보를 담고
다른 출처 요청에는 담지 않는 기본값은 무엇인가?



A.7

[주관식]

fetch의 credentials 옵션 중,
같은 출처 요청에는 인증 정보를 담고
다른 출처 요청에는 담지 않는 기본값은 무엇인가?

정답 : same-origin

별도 설정이 없으면 자격 증명 요청에서도
쿠키 등 인증 정보는 절대로 자동 전송되지 않는다.

Q.8

[객관식]

다음은 서로 다른 출처 간 리소스를 허용하는 ‘CORS 정책’과 ‘서버 응답 설정’에 대한 설명입니다.
빈칸 (A)와 (B)에 들어갈 알맞은 용어를 짝지은 것을 고르시오.

<보기>

CORS는 서로 다른 출처라도 서버가 허락한 범위 안에서 리소스를 주고받을 수 있게 해주는 정책입니다.
브라우저는 요청 시 자신의 출처 정보인 (A) 헤더를 담아 보내고, 서버는 이 요청을 허용할 출처를 (B) 헤더에 담아 응답합니다.

이때 서버에서 (B) 값으로 와일드카드(*)를 설정하면 출처에 상관없이 모든 요청이 접근 가능해져
보안에 취약해지므로, 실무에서는 허용할 출처를 명시적으로 지정하는 것이 더 안전합니다.

Q.8

[객관식]

다음은 서로 다른 출처 간 리소스를 허용하는 ‘CORS 정책’과 ‘서버 응답 설정’에 대한 설명입니다.
빈칸 (A)와 (B)에 들어갈 알맞은 용어를 짝지은 것을 고르시오.

<보기>

CORS는 서로 다른 출처라도 서버가 허락한 범위 안에서 리소스를 주고받을 수 있게 해주는 정책입니다. 브라우저는 요청 시 자신의 출처 정보인 (A) 헤더를 담아 보내고, 서버는 이 요청을 허용할 출처를 (B) 헤더에 담아 응답합니다.

이때 서버에서 (B) 값으로 와일드카드(*)를 설정하면 출처에 상관없이 모든 요청이 접근 가능해져 보안에 취약해지므로, 실무에서는 허용할 출처를 명시적으로 지정하는 것이 더 안전합니다.

- ① (A) Origin , (B) Access-Control-Allow-Origin
- ② (A) Access-Control-Request-Headers , (B) Origin
- ③ (A) Origin , (B) Access-Control-Request-Headers
- ④ (A) Access-Control-Request-Methods , (B) Origin

A.8

[객관식]

다음은 서로 다른 출처 간 리소스를 허용하는 ‘CORS 정책’과 ‘서버 응답 설정’에 대한 설명입니다.
빈칸 (A)와 (B)에 들어갈 알맞은 용어를 짝지은 것을 고르시오.

<보기>

CORS는 서로 다른 출처라도 서버가 허락한 범위 안에서 리소스를 주고받을 수 있게 해주는 정책입니다. 브라우저는 요청 시 자신의 출처 정보인 (A) 헤더를 담아 보내고, 서버는 이 요청을 허용할 출처를 (B) 헤더에 담아 응답합니다.

이때 서버에서 (B) 값으로 와일드카드(*)를 설정하면 출처에 상관없이 모든 요청이 접근 가능해져 보안에 취약해지므로, 실무에서는 허용할 출처를 명시적으로 지정하는 것이 더 안전합니다.

- ① (A) Origin , (B) Access-Control-Allow-Origin
- ② (A) Access-Control-Request-Headers , (B) Origin
- ③ (A) Origin , (B) Access-Control-Request-Headers
- ④ (A) Access-Control-Request-Methods , (B) Origin

CORS 동작 시 브라우저는 요청을 보낸 출처인 ‘Origin’ 헤더와 서버가 응답으로 돌려준 ‘Access-Control-Allow-Origin’헤더를 비교하여 접근을 제어합니다. 보안 강화를 위해서는 와일드카드(*) 대신 수락할 특정 출처 주소를 직접 명시하는 대응법이 권장됩니다.

Q.9

[O/X 퀴즈]

프록시 서버를 사용하면,
웹 애플리케이션이 리소스를 직접 요청하지 않고
같은 출처의 프록시 서버가 요청을 대신 전달하기 때문에
CORS 에러를 방지할 수 있다.



A.9

[O/X 퀴즈]

프록시 서버를 사용하면,
웹 애플리케이션이 리소스를 직접 요청하지 않고
같은 출처의 프록시 서버가 요청을 대신 전달하기 때문에
CORS 에러를 방지할 수 있다.

정답 : O

프록시 서버를 사용하면 웹 애플리케이션이 직접 API 요청하는 대신, 같은
출처에 위치한 프록시 서버를 통해 API를 요청하도록 구성할 수 있다.
따라서 웹 애플리케이션이 리소스와 동일한 출처에서 요청을 보내는 것처럼
보이기 때문에 CORS 에러를 방지할 수 있다.

Q.10

[객관식]

다음 중 같은 Origin(출처) 으로 올바른 것은 무엇일까요?

- ① `https://cotato.kr` ↔ `http://cotato.kr`
- ② `https://cotato.kr` ↔ `https://api.cotato.kr`
- ③ `https://cotato.kr:443` ↔ `https://cotato.kr`
- ④ `https://cotato.kr:3000` ↔ `https://cotato.kr:8080`

A.10

[객관식]

다음 중 같은 Origin(출처) 으로 올바른 것은 무엇일까요?

- ① `https://cotato.kr` ↔ `http://cotato.kr`
- ② `https://cotato.kr` ↔ `https://api.cotato.kr`
- ③ `https://cotato.kr:443` ↔ `https://cotato.kr`
- ④ `https://cotato.kr:3000` ↔ `https://cotato.kr:8080`

A.10

[객관식]

다음 중 같은 Origin(출처) 으로 올바른 것은 무엇일까요?

Origin은 프로토콜(Protocol), 호스트(Host), 포트(Port) 세 요소로 결정됩니다.

① `https://cotato.kr` ↔ `http://cotato.kr`
프로토콜이 다르므로 다른 Origin

② `https://cotato.kr` ↔ `https://api.cotato.kr`
호스트(도메인)가 다르므로 다른 Origin

③ `https://cotato.kr:443` ↔ `https://cotato.kr`
HTTPS의 기본 포트인 443은 생략 가능하므로 같은 Origin

④ `https://cotato.kr:3000` ↔ `https://cotato.kr:8080`
포트 번호가 다르므로 다른 Origin